Python — Zero to Master

23 topics, grouped as Beginner → Intermediate → Advanced → Master. Work through the topics in order; type every code example yourself and finish the practice tasks.

Contents

Beginner (7 topics)

01 Introduction and Installation
02 Variables and Data Types
03 Operators and Input/Output
04 Conditions and Loops
05 Functions
06 Lists, Tuples, Sets, Dictionaries
07 Strings and String Methods

Intermediate (6 topics)

08 File Handling
09 Exception Handling
10 Modules, Packages and pip
11 Object-Oriented Programming
12 Iterators, Generators, Decorators
13 Standard Library Highlights

Advanced (6 topics)

14 Type Hints and Static Checking
15 Concurrency: Threads, Processes, asyncio
16 Working with APIs and Databases
17 Testing with pytest
18 Web Frameworks Overview
19 Data Analysis: NumPy and Pandas

Master (4 topics)

20	Packaging and Project Structure
21	Performance and Profiling
22	Design Patterns and Clean Code
23	Real Projects

Beginner Level

01. Introduction and Installation

DEFINITION

Python is a readable, general-purpose language used for web, automation, data science and AI. Install Python 3 and run `python file.py`.

SYNTAX

```
python file.py
```

EXAMPLE

```
print('Hello, Python!')
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Python reads like plain English, like giving simple instructions to a helper in Tamil; that is why it is the first language for many.

தமிழ்

Python எளிய ஆங்கிலம் போலப் படிக்கக் கிடைப்பதால் உதவியாளருக்கு எளிய தமிழில் கட்டளை தருவது போன்றது; அதனால் பலருக்கு முதல் மொழி.

PROS

- Easy to learn
- Huge libraries (AI, web, data)
- Great community

CONS

- Slower than compiled languages
- GIL limits threads
- Mobile support is weak

ADVANCED / NEW FEATURES

- Python 3.14: free-threaded build officially supported, template strings (t-strings), deferred annotations
- Faster interpreter and experimental JIT (3.13+)

02. Variables and Data Types

DEFINITION

Dynamic typing: `int`, `float`, `str`, `bool`, `None`. Use `type()`; f-strings format output.

SYNTAX

```
name = value
f'{name}'
```

EXAMPLE

```
name = 'Priya'
age = 22
print(f'{name} is {age}')
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Labelled jars in a kitchen where the label can stick on any jar; variables just point to values, no type declaration needed.

தமிழ்

லேபிள் எந்த ஜாடியிலும் ஒட்டக்கூடிய சமையலறை ஜாடிகள் போல variables மதிப்புகளைச் சுட்டிக்காட்டுகின்றன; type அறிவிக்க வேண்டாம்.

PROS

- No declaration boilerplate
- Dynamic typing is flexible

CONS

- Type errors appear at runtime

ADVANCED / NEW FEATURES

- Walrus `:=` (3.8)
- `match` statements (3.10)
- Type parameter syntax (3.12)

03. Operators and Input/Output

DEFINITION

Arithmetic (`//`, `%`, `**`), comparison, logical operators, `input()` and `print()`.

SYNTAX

```
x = int(input())
```

EXAMPLE

```
n = int(input('Number: '))
print(n ** 2)
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A shop clerk asks the customer the quantity and prints the bill; input/output lets programs talk to people.

தமிழ்

கடை எழுத்தர் வாடிக்கையாளரிடம் அளவு கேட்டு பில் அச்சிடுவது போல input/output நிரல்கள் மனிதர்களுடன் பேச உதவுகிறது.

PROS

- Simple I/O
- Rich formatting

CONS

- Input is always text (convert it)

ADVANCED / NEW FEATURES

- Improved REPL in 3.13
- Better error messages

04. Conditions and Loops

DEFINITION

`if/elif/else`, `for`, `while`, `range`, `break`, `continue`, `enumerate`, `zip`.

SYNTAX

```
for x in seq:
while cond:
if / elif / else
```

EXAMPLE

```
for i, fruit in enumerate(['apple', 'mango'], start=1):
    print(i, fruit)
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A bus conductor goes seat by seat checking tickets; `for` loops over every item.

தமிழ்

நடத்துநர் இருக்கை இருக்கையாக டிக்கெட் சோதிப்பது போல `for` ஒவ்வொரு உருப்படியையும் சுற்றுகிறது.

PROS

- Readable indentation-based blocks
- Powerful iteration tools

CONS

- Indentation errors confuse beginners

ADVANCED / NEW FEATURES

- Structural pattern matching (3.10)

05. Functions

DEFINITION

Define with `def`; default, keyword and `*args/**kwargs` parameters; lambdas; docstrings.

SYNTAX

```
def name(a, b=1, *args, **kw):
    return x
```

EXAMPLE

```
def greet(name, greeting='Hello'):
    return f'{greeting}, {name}'
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A tea machine: put milk and tea powder, get tea; same machine for many cups.

தமிழ்

தேநீர் இயந்திரம்: பால், தூள் போட்டால் தேநீர்; ஒரே இயந்திரம் பல கோப்பைகளுக்கு.

PROS

- Simple function definition
- Default and keyword args

CONS

- Mutable default arguments pitfall

ADVANCED / NEW FEATURES

- Positional-only `/` parameters (3.8)
- `type` aliases (3.12)

06. Lists, Tuples, Sets, Dictionaries

DEFINITION

Lists are mutable sequences, tuples immutable, sets unique values, dicts key-value maps. Comprehensions create them concisely.

SYNTAX

```
[x for x in seq]
{k: v for ...}
```

EXAMPLE

```
squares = {n: n * n for n in range(5)}
evens = [n for n in range(10) if n % 2 == 0]
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A shopping list (list), a unique ration-card number set (set), a phone book (dict); Python collections fit each need.

தமிழ்

கடைப் பட்டியல் (list), தனித்துவ ரேஷன் எண்கள் (set), தொலைபேசி புத்தகம் (dict) போல Python collections ஒவ்வொரு தேவைக்கும் பொருந்தும்.

PROS

- Built-in powerful structures
- Comprehensions are concise

CONS

- Mutable aliasing bugs
- Lists are slower than NumPy arrays for numbers

ADVANCED / NEW FEATURES

- Dict ordering guaranteed (3.7)
- Union `|` for dicts (3.9)

07. Strings and String Methods

DEFINITION

Slicing, `split`, `join`, `strip`, `replace`, `find`, formatting.

SYNTAX

```
s.split(',')
s[::-1]
', '.join(items)
```

EXAMPLE

```
'a,b,c'.split(',')   # ['a', 'b', 'c']
'python'[::-1]       # 'nohtyp'
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Reading a name written on a banner letter by letter; strings can be sliced, split and joined.

தமிழ்

பேனரில் எழுதிய பெயரை எழுத்து எழுத்தாக வாசிப்பது போல strings-ஐ வெட்டலாம், பிரிக்கலாம், இணைக்கலாம்.

PROS

- Powerful text handling
- Unicode friendly

CONS

- Immutable (new object each change)

ADVANCED / NEW FEATURES

- `removeprefix/removesuffix` (3.9)
- f-string improvements (3.12)

Intermediate Level

08. File Handling

DEFINITION

Use `with open()` for safe file access; work with CSV and JSON modules.

SYNTAX

```
with open('f') as f:
```

EXAMPLE

```
with open('data.txt', 'w') as f:
    f.write('hello')
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Opening and closing the shop shutter safely every day; `with` closes files automatically.

தமிழ்

தினமும் கடை ஷட்டரை பாதுகாப்பாகத் திறந்து மூடுவது போல `with` கோப்புகளை தானாக மூடுகிறது.

PROS

- Safe resource handling
- Easy CSV/JSON modules

CONS

- Large files need streaming

ADVANCED / NEW FEATURES

- `pathlib` modern file APIs
- Better `open` error messages

09. Exception Handling

DEFINITION

`try/except/else/finally`, raise exceptions, custom exception classes.

SYNTAX

```
try:
except E as e:
finally:
```

EXAMPLE

```
try:
    x = int('abc')
except ValueError as e:
    print('Bad number', e)
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

If a customer's UPI payment fails, you switch to cash instead of cancelling the sale.

தமிழ்

வாடிக்கையாளரின் UPI தோல்வியுற்றால் விற்பனையை ரத்து செய்யாமல் ரொக்கத்துக்கு மாறுவது போல.

PROS

- Clear error flow
- Custom exceptions

CONS

- Broad `except` hides bugs

ADVANCED / NEW FEATURES

- ExceptionGroup and `except*` (3.11)
- Exception notes

10. Modules, Packages and pip

DEFINITION

Import code from files and packages; manage dependencies with `pip` and virtual environments.

SYNTAX

```
python -m venv .venv
pip install pkg
```

EXAMPLE

```
python -m venv .venv
source .venv/bin/activate
pip install requests
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Each family keeps its own kitchen utensils to avoid mix-ups; virtual environments isolate project libraries.

தமிழ்

குழப்பம் தவிர்க்க ஒவ்வொரு குடும்பமும் தங்கள் பாத்திரங்களை தனியாக வைப்பது போல virtual environments திட்ட நூலகங்களை தனிமைப்படுத்துகின்றன.

PROS

- Isolated dependencies
- Huge PyPI ecosystem

CONS

- Packaging tools are fragmented

ADVANCED / NEW FEATURES

- `uv` fast package manager
- `pyproject.toml` standard
- PEP 668 externally-managed environments

11. Object-Oriented Programming

DEFINITION

Classes, `__init__`, inheritance, polymorphism, `@property`, dunder methods, dataclasses.

SYNTAX

```
class A(B):
    def __init__(self): ...
```

EXAMPLE

```
from dataclasses import dataclass
@dataclass
class Point:
    x: int
    y: int
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A house plan and the houses built from it; classes model real things.

தமிழ்

வீட்டு வரைபடமும் அதிலிருந்து கட்டிய வீடுகளும் போல classes உண்மையான பொருட்களை மாதிரியாக்குகின்றன.

PROS

- Clear modelling
- Dataclasses reduce boilerplate

CONS

- No true private members
- Multiple inheritance complexity

ADVANCED / NEW FEATURES

- `dataclass(slots=True)`
- `@override` decorator (3.12)

12. Iterators, Generators, Decorators

DEFINITION

Generators `yield` values lazily; decorators wrap functions to add behaviour.

SYNTAX

```
def gen():
    yield x
@decorator
```

EXAMPLE

```
def timer(fn):
    def wrapper(*a, **k):
        print('calling', fn.__name__)
        return fn(*a, **k)
    return wrapper
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A token machine giving numbers on demand; a decorator is gift wrapping that adds a bow to any function.

தமிழ்

கேட்கும்போது எண் தரும் டோக்கன் இயந்திரம்; decorator எந்த function-க்கும் அலங்கார ரிப்பன் சேர்க்கும் பரிசுச் சுற்றல் போன்றது.

PROS

- Memory-efficient iteration
- Clean cross-cutting logic

CONS

- Harder to debug

ADVANCED / NEW FEATURES

- Generator expressions
- `yield from`

13. Standard Library Highlights

DEFINITION

`datetime`, `pathlib`, `collections`, `itertools`, `re`, `json`, `logging`, `argparse`.

SYNTAX

```
from collections import Counter
```

EXAMPLE

```
from collections import Counter
Counter('banana').most_common(1)
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A toolbox that ships with the house; Python's standard library has tools for dates, files and maths.

தமிழ்

வீட்டுடன் வரும் கருவிப்பெட்டி போல Python standard library-இல் தேதி, கோப்பு, கணிதக் கருவிகள் உள்ளன.

PROS

- Batteries included
- No installs

CONS

- Some modules are old-style

ADVANCED / NEW FEATURES

- `tomllib` (3.11)
- `zoneinfo` (3.9)
- `itertools.batched` (3.12)

Advanced Level

14. Type Hints and Static Checking

DEFINITION

Annotate with `list[int]`, `dict[str, int]`, `Optional`; check with mypy/pyright.

SYNTAX

```
def f(x: int) -> str:
```

EXAMPLE

```
def total(nums: list[int]) -> int:
    return sum(nums)
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Writing the weight unit on each parcel so mistakes are caught; type hints document expected types.

தமிழ்

ஒவ்வொரு பார்சலிலும் எடை அலகை எழுதுவது போல type hints எதிர்பார்க்கும் வகைகளை ஆவணப்படுத்துகின்றன.

PROS

- Better editor help
- Catches bugs with mypy/pyright

CONS

- Not enforced at runtime

ADVANCED / NEW FEATURES

- `X
- Y` unions (3.10)
- `Self`, `TypedDict` improvements
- Deferred annotations (3.14)

15. Concurrency: Threads, Processes, asyncio

DEFINITION

Threads suit I/O, processes suit CPU work, `asyncio` handles many I/O tasks with `async/await`.

SYNTAX

```
async def f(): await g()
```

EXAMPLE

```
import asyncio
async def main():
    await asyncio.gather(asyncio.sleep(1), asyncio.sleep(1))
asyncio.run(main())
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A waiter taking many orders and serving as food gets ready instead of waiting at one table.

தமிழ்

ஒரு மேசையில் காத்திருக்காமல் உணவு தயாராகும்போது பல ஆர்டர்களை எடுத்து சேவை செய்யும் சர்வர் போல.

PROS

- High I/O concurrency
- Simple async syntax

CONS

- Mixing sync and async is tricky

ADVANCED / NEW FEATURES

- Free-threaded Python (no GIL)
- `asyncio.TaskGroup` (3.11)

16. Working with APIs and Databases

DEFINITION

`requests` / `httpx` for HTTP; `sqlite3`, SQLAlchemy for databases.

SYNTAX

```
requests.get(url)
engine = create_engine(...)
```

EXAMPLE

```
import requests
r = requests.get('https://api.github.com')
print(r.status_code)
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Calling a supplier on the phone and noting the order in a ledger; Python talks to APIs and databases easily.

தமிழ்

சப்ளையரை அழைத்து ஆர்டரை பதிவேட்டில் குறிப்பது போல Python API-கள், databases-உடன் எளிதில் பேசுகிறது.

PROS

- Many mature clients
- Easy JSON handling

CONS

- Blocking calls need async clients

ADVANCED / NEW FEATURES

- `httpx` modern client
- SQLAlchemy 2.0 style

17. Testing with pytest

DEFINITION

Write small test functions, fixtures and parametrised tests.

SYNTAX

```
def test_x():
    assert f() == 1
```

EXAMPLE

```
def test_add():
    assert add(2, 3) == 5
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A teacher's answer-key check of exercises; pytest verifies functions automatically.

தமிழ்

ஆசிரியர் விடைத்தாள் சரிபார்ப்பது போல pytest functions-ஐ தானாகச் சரிபார்க்கிறது.

PROS

- Simple tests
- Powerful fixtures and plugins

CONS

- Over-mocking makes tests brittle

ADVANCED / NEW FEATURES

- pytest 8 features
- Hypothesis property testing

18. Web Frameworks Overview

DEFINITION

Flask (micro), Django (batteries included), FastAPI (modern async APIs).

SYNTAX

```
pip install flask django fastapi
```

EXAMPLE

```
pip install fastapi uvicorn
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Different shop styles: a small kiosk (Flask), a supermarket (Django), a fast delivery counter (FastAPI).

தமிழ்

வெவ்வேறு கடை வகைகள்: சிறிய கியோஸ்க் (Flask), பல்பொருள் அங்காடி (Django), வேகமான டெலிவரி கவுண்டர் (FastAPI).

PROS

- Frameworks for every need
- Great docs

CONS

- Choosing can be confusing

ADVANCED / NEW FEATURES

- Django 5.x async support
- FastAPI popularity growth

19. Data Analysis: NumPy and Pandas

DEFINITION

Vectorised arrays and DataFrames for cleaning and analysing data.

SYNTAX

```
df.groupby('city')['sales'].sum()
```

EXAMPLE

```
import pandas as pd
df = pd.read_csv('sales.csv')
df.groupby('city')['amount'].sum()
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A shop owner's monthly sales sheet with totals by area; pandas does it in one line.

தமிழ்

கடை உரிமையாளரின் மாதாந்திர விற்பனைத் தாள் பகுதிவாரியான மொத்தம் போல pandas ஒரே வரியில் செய்கிறது.

PROS

- Fast analysis
- Rich ecosystem

CONS

- High memory use
- Slow for huge data (use Polars/DuckDB)

ADVANCED / NEW FEATURES

- pandas 2/3 with Arrow
- Polars and DuckDB alternatives

Master Level

20. Packaging and Project Structure

DEFINITION

`pyproject.toml`, src layout, linting (ruff), formatting (black), pre-commit, publishing to PyPI.

SYNTAX

```
pyproject.toml
ruff check .
```

EXAMPLE

```
ruff check . && pytest
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Packing and labelling goods properly before they go to the market; packaging and linting prepare code for sharing.

தமிழ்

சந்தைக்கு அனுப்பும் முன் பொருட்களை சரியாகப் பேக் செய்து லேபிள் இடுவது போல packaging, linting குறியீட்டைப் பகிரத் தயார் செய்கின்றன.

PROS

- Reproducible installs
- Consistent style

CONS

- Tooling churn

ADVANCED / NEW FEATURES

- `uv`, `ruff`, `pyproject.toml` standard
- Trusted publishing on PyPI

21. Performance and Profiling

DEFINITION

Profile with `cProfile`, use built-ins, avoid premature optimisation, NumPy, caching with `functools.lru_cache`.

SYNTAX

```
python -m cProfile app.py
```

EXAMPLE

```
from functools import lru_cache
@lru_cache
def fib(n): return n if n < 2 else fib(n-1) + fib(n-2)
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

A doctor measures pulse before treating; profile code before optimising.

தமிழ்

சிகிச்சைக்கு முன் மருத்துவர் நாடி பார்ப்பது போல மேம்படுத்தும் முன் குறியீட்டை profile செய்யுங்கள்.

PROS

- Targeted speed-ups
- Simple tools

CONS

- Premature optimisation wastes time

ADVANCED / NEW FEATURES

- Specializing adaptive interpreter (3.11+)
- Experimental JIT (3.13+)
- Free-threading

22. Design Patterns and Clean Code

DEFINITION

PEP 8, SOLID, composition over inheritance, dependency injection, context managers.

SYNTAX

```
with Timer():
    ...
```

EXAMPLE

```
class Timer:
    def __enter__(self): ...
    def __exit__(self, *exc): ...
```

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Clean code is like a tidy kitchen: everything has its place so anyone can cook.

தமிழ்

சுத்தமான குறியீடு ஒழுங்கான சமையலறை போல; எல்லாவற்றுக்கும் இடம் இருப்பதால் யாரும் சமைக்கலாம்.

PROS

- Readable, maintainable code
- PEP 8 consistency

CONS

- Style debates

ADVANCED / NEW FEATURES

- Ruff formatter
- Pattern matching enables clearer branching

23. Real Projects

DEFINITION

CLI expense tracker, web scraper, REST API, data dashboard, automation scripts with scheduling.

REAL-WORLD EXAMPLE (ENGLISH + தமிழ்)

English

Expense tracker, scraper and API are classic practice projects that combine many topics.

தமிழ்

செலவு கண்காணிப்பு, scraper, API ஆகியவை பல தலைப்புகளை இணைக்கும் பாரம்பரிய பயிற்சி project-கள்.

PROS

- Real practice

CONS

- Takes time

ADVANCED / NEW FEATURES

- Use `uv`, FastAPI, pytest, ruff

Interview Question Bank

Q1. List vs tuple?
Lists are mutable; tuples are immutable and hashable (if contents are).

Q2. What is a decorator?
A function that takes a function and returns an enhanced one, applied with `@`.

Q3. `is` vs `==`?
`==` compares values; `is` compares object identity.

Q4. What are generators?
Functions with `yield` that produce values lazily, saving memory.

Q5. Explain the GIL.
CPython's global interpreter lock lets one thread run Python bytecode at a time; use processes or async for scaling.

Q6. Mutable default argument pitfall?
Defaults are evaluated once; `def f(x=[])` shares the list between calls - use `None`.

Q7. What is a context manager?
An object with `__enter__`/`__exit__` used by `with` for setup/cleanup.

Q8. `*args` and `**kwargs`?
Collect extra positional and keyword arguments.

Q9. List comprehension vs generator expression?
A list comprehension builds a list now; a generator expression yields items lazily.

Q10. What is a virtual environment?
An isolated Python environment with its own installed packages.

Capstone Project Ideas

1. CLI expense tracker
2. Web scraper with BeautifulSoup
3. Weather app using API
4. Automated email sender
5. Data analysis dashboard